Derivatives of a Language with Respect to a *Letter*

A useful concept in automating lexers is *derivative* of a language.

Let $L \subseteq A^*$ be a language and let $x \in A$ be a letter. Then

$$\partial_x L = \{w \in A^* \mid xw \in L\}$$

Example: if $L = \{\varepsilon, b, aa, abb, bbba\}$ then

$$\partial_a L = \{a, bb\}$$

$$\partial_b L = \{\varepsilon, bba\}$$

Observe that ∂_x ignores all words that do not begin by x and collects the suffixes of those that begin with x.
If $A = \{a, b\}$ and $L \subseteq A^*$, we can decompose L into disjoint sets:

$$L = (a\,\partial_a L) \cup (b\,\partial_b L) \cup \{\varepsilon \mid nullable(L)\}$$

Derivatives ignore ε if it is in L; the last part puts it back iff L was nullable.

Derivatives of a Language with Respect to a *Word*

Let us generalize the definition from a letter to deriving with respect to an arbitrary word (sequence of letters).

Let $L \subseteq A^*$ be a language and let $v \in A$ be a *word*. Define:

$$\partial_v L = \{w \in A^* \mid vw \in L\}$$

By definition,

$$w \in \partial_v L \iff vw \in L$$

Because $w \in \partial_\varepsilon L$ means $\varepsilon w \in L$, we have $\partial_\varepsilon L = L$. Moreover,

$$w \in \partial_{av} L \iff avw \in L \iff vw \in \partial_a L \iff w \in \partial_v(\partial_a L)$$

Therefore, $\partial_{av} L = \partial_v(\partial_a L)$, which gives a recursive definition of derivative with respect to a word.

Rules for Derivatives of Regular Expressions

From the meaning of regular expression and the definition of derivative with respect to a letter $a \in A$, we obtain the following derivative rules for regular expressions (remember that $+$ denotes union):

$$
\begin{aligned}
\partial_a \varepsilon &= \emptyset \\
\partial_a \emptyset &= \emptyset \\
\partial_a b &= \begin{cases} \varepsilon & \text{if } b = a \\ \emptyset & \text{if } b \in A \setminus \{a\} \end{cases} \\
\partial_a (r_1 + r_2) &= (\partial_a r_1) + (\partial_a r_2) \\
\partial_a (r_1\ r_2) &= \begin{cases} (\partial_a r_1\ r_2) + \partial_a r_2 & \text{if } nullable(r_1) \\ \partial_a r_1\ r_2 & \text{otherwise} \end{cases} \\
\partial_a (r^*) &= (\partial_a r)\ r^*
\end{aligned}
$$

Derivative of a regular expression is a regular expression!

Checking if Word Belongs to a Regular Expression

$$w \in L \iff w\varepsilon \in L \iff \varepsilon \in \partial_w L \iff nullable(\partial_w L)$$

If $w = w_1 \dots w_n$ where $w_1, \dots, w_n \in A$, then

$$w_1 \dots w_n \in L \iff \partial_{w_1 \dots w_n} L \iff nullable(\partial_{w_n} \dots \partial_{w_1} L)$$

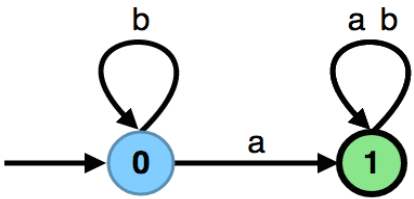
When L is given by a regular expression, we can check the last condition using an algorithm:

- compute derivative of L with respect to letters of the word, starting from the first letter (at each step we have a regular expression)
- at the end, we check if the final regular expression is nullable (we have seen a recursive algorithm for that in the previous lecture)

However, without further optimizations, this can be inefficient. ZipLex has those optimizations, so it works well. However, for completeness we review a classic approach based on automata.

Finite Automaton (Finite State Machine)

$A = (\Sigma, Q, q_0, \delta, F)$



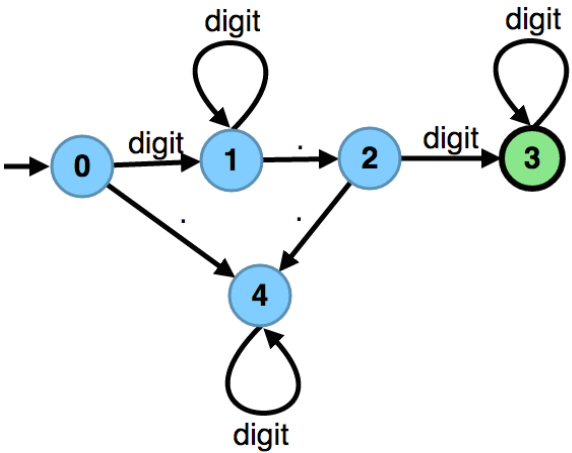
$$\delta \subseteq Q \times \Sigma \times Q,$$
$$q_0 \in Q,$$
$$F \subseteq Q$$

$$q_0 \in Q$$
$$q_1 \subseteq Q$$
$$\delta = \{ (q_0, a, q_1), (q_0, b, q_0), (q_1, a, q_1), (q_1, b, q_1), \}$$

- Σ - alphabet
- Q - states (nodes in the graph)
- q_0 - initial state (with '->' sign in drawing)
- δ - transitions (labeled edges in the graph)
- F - final states (double circles)

Numbers with Decimal Point

digit digit* . digit digit*

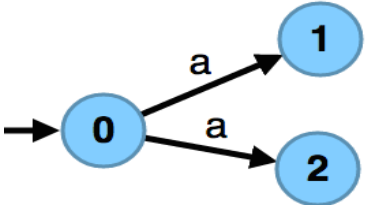


What if the decimal part is optional?

Kinds of Finite State Automata

- DFA: δ is a function : $(Q, \Sigma) \mapsto Q$
- NFA: δ could be a relation

- In NFA there is no unique next state. We have a set of possible next states.



Remark: Relations and Functions

- Relation $r \subseteq B \times C$

  $r = \{ \ldots, (b,c1), (b,c2), \ldots \}$

- Corresponding function: $f : B \to 2^C$

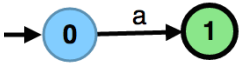
  $f = \{ \ldots (b,\{c1,c2\}) \ldots \}$

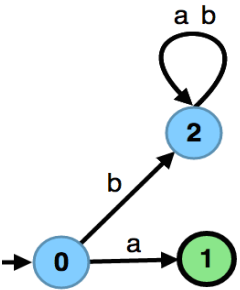
  $f(b) = \{ c \mid (b,c) \in r \}$

- Given a state, next-state function returns a **set** of new states

  for *deterministic* automaton, set has *exactly 1* element

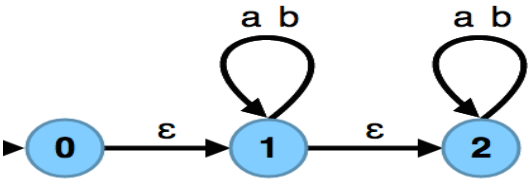
Allowing Undefined Transitions





- Undefined transitions are equivalent to transition into a sink state (from which one cannot recover)

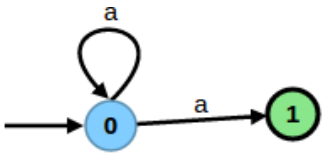
Allowing Epsilon Transitions



- Epsilon transitions:
  - traversing them does not consume anything
- Transitions labeled by a word:
  - traversing them consumes the entire word

When Automaton Accepts a Word

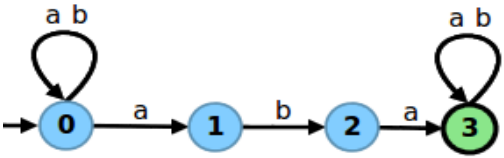
Automaton accepts a word *w* iff there **exists a path** in the automaton from the starting state to some accepting state such that concatenation of words on the path gives *w*.



- Does the automaton accept the word ***a*** ?

Exercise

- Construct a NFA that recognizes all strings over {a,b} that contain "aba" as a substring



Running NFA (without epsilons)

```
def δ(a : Char)(q : State) : Set[States] = { ... }
def δ'(a : Char, S : Set[States]) : Set[States] = {
  for (q1 <- S, q2 <- δ(a)(q1)) yield q2  // S.flatMap(δ(a))
}
def accepts(input : MyStream[Char]) : Boolean = {
  var S : Set[State] = Set(q0) // current set of states
  while (!input.EOF) {
    val a = input.current
    S = δ'(a,S)          // next set of states
  }
  !(S.intersect(finalStates).isEmpty)
}
```

NFA Vs DFA

- Every DFA is also a NFA (they are a special case)
- For every NFA there exists an equivalent DFA that accepts the same set of strings
- But, NFAs could be exponentially smaller (succinct)
- There are NFAs such that **every** DFA equivalent to it has exponentially more number of states

Regular Expressions and Automata

Theorem:

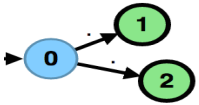
Let L be a language. There exists a regular expression that describes it if and only if there exists a finite automaton that accepts it.

Algorithms:

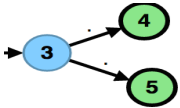
- regular expression → automaton (important!)
- automaton → regular expression (cool)

Recursive Constructions

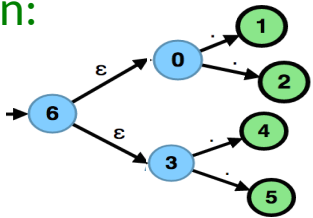
A_1:



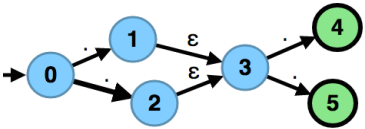
A_2:



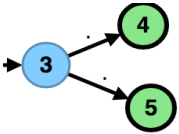
Union:



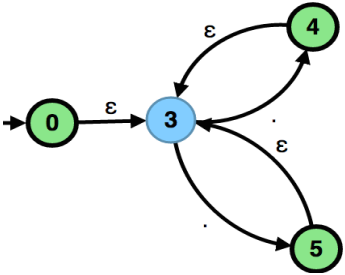
Concatenation:



Recursive Constructions

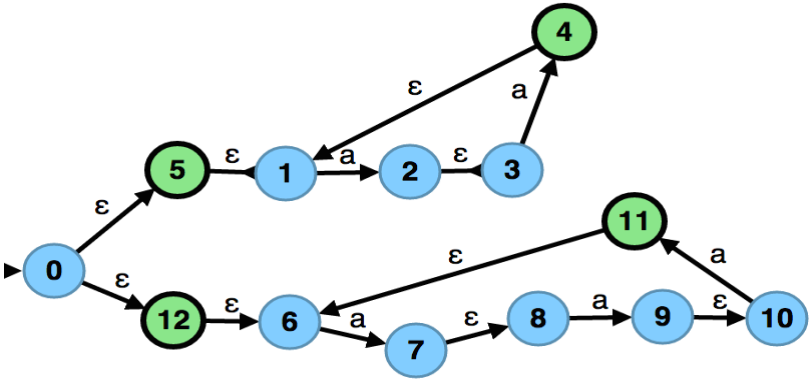


Star:



Exercise: (aa)* | (aaa)*

- Construct an NFA for the regular expression

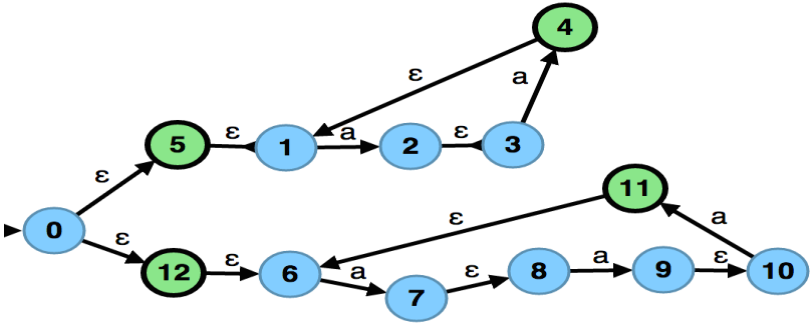


NFAs to DFAs (Determinization)

- keep track of a set of all possible states in which the automaton could be

- view this finite set as one state of new automaton

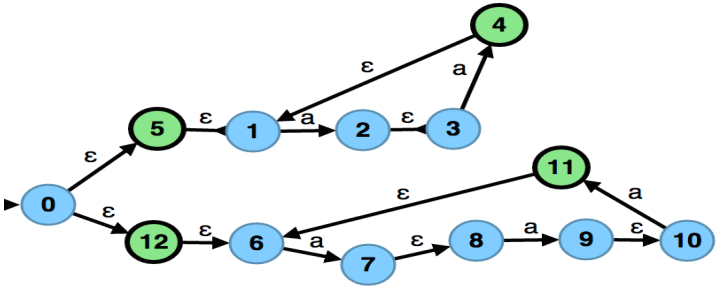
NFA to DFA Conversion



Possible states of the DFA: 2^Q

{ { } , { 0},...{12}, {0,1}, ...,{0,12}, ...{12, 12},
{0,1,2} ..., { 0,1,2...,12 } }

NFA to DFA Conversion



Epsilon Closure

–All states reachable from a state through epsilon

– q ∈ $E(q)$

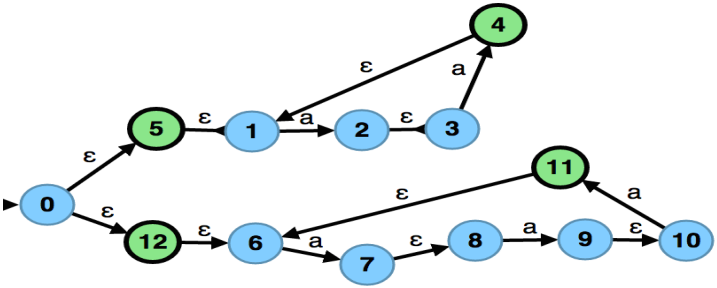
– If $q_1 \in E(q)$ and $\delta(q_1, \epsilon, q_2)$ then $q_2 \in E(q)$

E(0) = { } E(1) = { } E(2) = { }

NFA to DFA Conversion

- DFA: $(\Sigma, 2^Q, q_0', \delta', F')$
- $q_0' = E(q_0)$
- $\delta'(q', a) = \bigcup_{\{\exists q_1 \in q',\, \delta(q_1, a, q_2)\}} E(q_2)$
- $F' = \{\, q' | q' \in 2^Q, q' \cap F \neq \emptyset\}$

NFA to DFA Conversion through Examle



Clarifications

- what happens if a transition on an alphabet ‘a’ is not defined for a state ‘q’ ?
- $\delta'(\{q\}, a) = \emptyset$
- $\delta'(\emptyset, a) = \emptyset$

- Empty set represents a state in the NFA
- It is a trap/sink state: a state that has self-loops for all symbols, and is non-accepting.

Relationship between Automata and Derivatives

Regular expressions can be viewed as states of a deterministic automaton.

For $a \in A$,
$s' = \delta(s, a)$ corresponds to $r' = \partial_a s$.

Problem: there can be an infinite number of syntactically different regular expressions, so the automaton is infinite (states may get bigger and bigger).

Solution: using properties of regular expression (e.g. $+$ is associative and commutative), we can identify those expressions that denote the same language. This gives back a finite state machine.

This is the essence of ZipLex tool that we use in the labs.

Grammars

Expressive Power of Automata

For which of the following languages can you find an automaton or regular expression:

- Sequence of open or closed parentheses of even length? E.g. (), ((,)),)()))(, …
- as many digits before as after decimal point?
- Sequence of balanced parentheses

  ((()) ()) - balanced

  ( )) (( ) - not balanced
- Comments from // until LF
- Nested comments like /* … /* */ … */

Expressive Power of Automata

For which of the following languages can you find an automaton or regular expression:

- Sequence of open or closed parentheses of even length? E.g. (), ((,)),)()))(, … yes
- as many digits before as after decimal point? No
- Sequence of balanced parentheses
  ((()) ()) - balanced
  ( )) (( ) - not balanced No
- Comments from // until LF Yes
- Nested comments like /* … /* */ … */ No

Automaton that Claims to Recognize $\{ a^n b^n \mid n >= 0 \}$

Make the automaton deterministic

Let the resulting DFA have K states, $|Q|=K$

Feed it a, aa, aaa, ... Let q_i be state after reading a^i

$q_0, q_1, q_2, \ldots, q_K$

This sequence has length K+1 -> a state must repeat

$q_i = q_{i+p}$ $\quad\quad$ $p > 0$

Then the automaton should accept $a^{i+p}b^{i+p}$.

But then it must also accept

$$a^i\, b^{i+p}$$

because it is in state after reading a^i as after a^{i+p}.

So it does not accept the given language.

Limitations of Regular Languages

- Every automaton can be made deterministic
- Automaton has finite memory, cannot count
- Deterministic automaton from a given state behaves always the same
- If a string is too long, deterministic automaton will repeat its behavior

Pumping Lemma

If L is a regular language, then there exists a positive integer p (the pumping length) such that every string $s \in L$ for which $|s| \geq p$, can be partitioned into three pieces, $s = x\,y\,z$, such that

- $|y| > 0$
- $|xy| \leq p$
- $\forall i \geq 0.\ xy^iz \in L$

Let's try again: $\{ a^n b^n \mid n >= 0 \}$

Finite State Automata are Limited

Let us use (context-free) **grammars**!

Context Free Grammar for $a^n b^n$

S ::= ε - first rule of this grammar

S ::= a S b - second rule of this grammar.

Example of a derivation

S => aSb => a aSb b => aa aSb bb => aaabbb

Parse tree: leaves give us the result

Context-Free Grammars

G = (A, N, S, R)

- A - terminals (alphabet for generated words $w \in A^*$)
- N - non-terminals – symbols with (recursive) definitions
- Grammar rules in R are pairs (n,v), written

  n ::= v where

  $n \in N$ is a non-terminal

  $v \in (A \cup N)^*$ - sequence of terminals and non-terminals

A derivation in G starts from the starting symbol S

- Each step replaces a non-terminal with one of its right hand sides

Example from before: G = ({a,b}, {S}, S, {(S,ε), (S,aSb)})

Parse Tree

Given a grammar G = (A, N, S, R), t is a **parse tree** of G
iff t is a node-labelled tree with ordered children that satisfies:

- root is labeled by S
- leaves are labelled by elements of A
- each non-leaf node is labelled by an element of N
- for each non-leaf node labelled by n whose children left to right are labelled by $p_1 \ldots p_n$, we have a rule $(n ::= p_1 \ldots p_n) \in R$

Yield of a parse tree t is the unique word in A^* obtained by reading the leaves of t from left to right

Language of a grammar G = words of all yields of parse trees of G

$L(G) = \{yield(t) \mid isParseTree(G,t)\}$

$w \in L(G) \quad \Leftrightarrow \quad \exists t.\ w=yield(t) \wedge isParseTree(G,t)$

isParseTree - **easy** to check condition, given t

Harder: know *if for a word there **exists** a parse tree*

Grammar Derivation

A **derivation** for G is any sequence of words $p_i \in (A \cup N)^*$, whose:

- first word is S
- each subsequent word is obtained from the previous one by replacing one of its letters by right-hand side of a rule in R :

  $p_i = unv$, $(n::=q) \in R$,

  $p_{i+1} = uqv$
- Last word has only letters from A

Each parse tree of a grammar has one or more derivations, which result in expanding tree gradually from S

- Different orders of expanding non-terminals may generate the same tree
- Leftmost derivation: always expands leftmost non-terminal
  - Rightmost derivation: always expands rightmost non-terminal

Remark

We abbreviate

S ::= p

S ::= q

as

S ::= p | q

Example: Parse Tree vs Derivation

Consider this grammar G = ({a,b}, {S,P,Q}, S, R) where R is:

S ::= PQ
P ::= a | aP
Q ::= ε | aQb

Show a parse tree for aaaabb
Show at least two derivations that correspond to that tree.

Balanced Parentheses Grammar

Consider the language L consisting of precisely those words consisting of parentheses "**(**" and "**)**" that are balanced (each parenthesis has the matching one)

- Example sequence of parentheses

  ((()) ()) - balanced, belongs to the language

  ( )) (( ) - not balanced, does not belong

Exercise: give the grammar and example derivation for the first string.

Balanced Parentheses Grammar

G_1 $S ::= \varepsilon \mid S(S)S$

G_2 $S ::= \varepsilon \mid (S)S$

G_3 $S ::= \varepsilon \mid S(S)$

G_4 $S ::= \varepsilon \mid S\,S \mid (S)$

These all define the same language, the language of balanced parentheses.